

Ubuntu / WSL Environment

AI Resume Screening System — Development and Execution Environment

1. Overview

Ubuntu, running through **Windows Subsystem for Linux (WSL)**, was the development and execution environment for the AI Resume Screening System. It provided the Linux terminal used to run the Python backend, manage the virtual environment, operate Git and, later, run the Flask API and the Streamlit dashboard.

The role of Ubuntu grew as the project grew:

- **Original role:** environment for developing and testing the Python resume-screening engine.
- **Later role:** also hosts the Flask API that n8n (running in Docker) calls, runs the Streamlit dashboard, and is the network endpoint that n8n connects to.

Related documents: **Python_Implementation.docx** and **Python_n8n_Integration_and_Workflow.docx**.

2. Why Ubuntu/WSL Was Selected

WSL provides a Linux environment directly on Windows, without a separate Linux machine or dual boot. It suited the project because:

- Python is well supported on Linux and packages are easy to install from the terminal
- The project needed Linux commands, virtual environments, Git/GitHub and command-line testing
- Backend and AI development is commonly done in Linux environments
- It allowed Linux usage to be demonstrated as part of the project

3. Initial Ubuntu/WSL Setup

In the original stage, Ubuntu was used for project management, Python execution, virtual environment management, Git/GitHub operations and command-line testing. The project was maintained in `~/resume_screener/python-engine`.

3.1 Accessing Windows files

The resume PDF was stored in Windows while Python ran inside WSL. Windows files were therefore accessed through the WSL mount path:

```
| /mnt/c/Users/.../Downloads/...
```

3.2 Git ignore rules

Files that should not be tracked (API key, virtual environment, cache and resume PDFs) were excluded using `.gitignore`:

```
| .env  
| venv/
```

```
__pycache__/  
*.pdf
```

3.3 Original testing sequence

```
Python execution  
↓  
PDF access  
↓  
PDF extraction  
↓  
Gemini API  
↓  
JSON output
```

Git was then used to commit and push the completed Python changes to GitHub.

4. Changes to the Environment Made Later

When the teammate integrated the Python engine with n8n, the Ubuntu environment took on additional responsibilities:

Change	Effect on Ubuntu/WSL
Flask API added	The backend is now started from Ubuntu with <code>python3 api.py</code> and listens on port 5000.
Streamlit dashboard added	The HR dashboard is started from Ubuntu with <code>streamlit run app.py</code> and opened at <code>http://localhost:8501</code> .
n8n running in Docker	Docker cannot reach the Flask server through <code>localhost</code> , so n8n uses the WSL IP address (172.19.95.137) with port 5000.
Result storage	Stored results are kept in <code>data/candidates.json</code> inside the project directory and can be inspected from the terminal.
Separate Git branch	The integration work was maintained on the branch <code>shivani-n8n-integration</code> .

5. Final Ubuntu/WSL Environment

5.1 Project location and files

```
/home/shivani/resume_screener/python-engine/  
├── api.py  
├── app.py  
├── extract.py  
├── score.py  
├── n8n_runner.py  
├── requirements.txt  
├── data/  
└── venv/
```

5.2 Virtual environment

A Python virtual environment keeps the project's dependencies separate from the system Python installation. It is activated with:

```
cd /home/shivani/resume_screener/python-engine
source venv/bin/activate
```

5.3 Running the Flask backend

```
python3 api.py
```

The backend runs on port **5000** and provides these endpoints:

- `/screen` — receives a resume and job description and processes them
- `/result` — receives the final structured screening result
- `/health` — additional endpoint provided by the API

5.4 Running the Streamlit dashboard

```
streamlit run app.py
```

The dashboard is opened in a browser at `http://localhost:8501`.

5.5 Git and GitHub

Ubuntu is used for version control. Commands such as `git status`, `git branch --show-current` and `git fetch origin` were used to check the repository status and current branch before pushing work to GitHub.

5.6 Docker and Ubuntu

n8n runs in a Docker container while the Python backend runs in Ubuntu. The two therefore act as separate environments that communicate over the network:

```
n8n (Docker)
  ↓ HTTP POST
http://172.19.95.137:5000/screen
  ↓
Flask API (Ubuntu/WSL)
```

6. Linux Commands Used

Command	Purpose
<code>cd</code>	Change directory
<code>ls</code>	List files
<code>pwd</code>	Display current directory
<code>cat</code>	Display file contents (for example, <code>cat data/candidates.json</code>)
<code>source</code>	Activate the virtual environment
<code>python3</code>	Run Python programs
<code>streamlit run</code>	Start the dashboard
<code>git status</code>	Check Git changes
<code>git branch</code>	Check Git branches
<code>git fetch</code>	Retrieve remote repository information
<code>rm</code>	Remove files

7. Problems Encountered and Solutions

Problem	Solution	Stage
Resume stored in Windows while Python ran in WSL	Accessed the file through the <code>/mnt/c/</code> mount path.	Original
Confusion about opening WSL project files through Windows File Explorer	Managed the project through WSL and VS Code.	Original
Unwanted files (<code>.env</code> , <code>venv/</code> , cache, PDFs) in Git	Excluded them using <code>.gitignore</code> .	Original
Python commands failed outside the correct directory or environment	Moved to the project directory and ran <code>source venv/bin/activate</code> first.	Later
Incorrect paths when accessing project files and stored results	Confirmed the real project location and used <code>/home/shivani/resume_screener/python-engine/</code> as the main directory.	Later
n8n in Docker could not reach Flask through <code>localhost</code>	Used the WSL IP address (<code>172.19.95.137</code>) with port 5000.	Later
Streamlit failed unless started correctly	Started it from the correct Python environment using <code>streamlit run app.py</code> .	Later

8. Testing and Validation

In the original stage, the Ubuntu environment was used to test Python execution, PDF access, PDF extraction, the Gemini API and JSON output. In the later stage, it was also used to run the Flask server and the Streamlit dashboard, and to inspect `data/candidates.json` from the terminal while debugging result storage.

9. My Contribution vs Later Team Work

My original contribution	Later modifications by teammate
Set up and used the Ubuntu/WSL development environment	Ran Flask and Streamlit from the environment for the full workflow
Python execution, virtual environment and Git/GitHub operations	Separate integration branch (<code>shivani-n8n-integration</code>)
Accessing Windows files through <code>/mnt/c/</code>	Docker-to-WSL connection using the WSL IP address
<code>.gitignore</code> configuration	Path corrections and inspecting <code>data/candidates.json</code> during debugging

10. Current Limitations

- Ubuntu/WSL is a development environment; the application runs only while it is started manually on a personal computer.
- The WSL IP address used by n8n is fixed in the workflow and may need updating if the network configuration changes.
- Flask and Streamlit are started from separate terminal sessions.

11. Future Scope

In a production version, the components could be moved from WSL to a Linux server or cloud-based Linux virtual machine. Possible improvements include:

- Hosting the Flask backend on a Linux server
- Running n8n continuously on a Linux server
- Using Docker containers for deployment
- Connecting a production database
- Automated backups
- HTTPS and security configuration
- Server monitoring and automated deployment

12. Final Outcome

Ubuntu/WSL provided the Linux environment in which the resume-screening engine was developed, tested and later run as part of the current prototype. It supported Python development, virtual environment management, Git operations, the Flask backend, the Streamlit dashboard and communication with n8n running in Docker.